```
/**
 * db.js — SQLite database layer
 *
 * Tables:
 *   players  — telegram_id ↔ wallet_address mapping
 *   games    — known game state (synced from chain events)
 *   secrets  — encrypted secret+salt stored server-side per player per game
 */

const Database = require('better-sqlite3');
const path     = require('path');
const crypto   = require('crypto');

const DB_PATH = process.env.DB_PATH || path.join(__dirname, 'cashx.db');
const SECRET_KEY = process.env.SECRET_ENCRYPTION_KEY; // 32-char hex key for AES-

let db;

// ─── Init ──────────────────────────────────────────────

function init() {
  db = new Database(DB_PATH);
  db.pragma('journal_mode = WAL');
  db.pragma('foreign_keys = ON');

  db.exec(`
    CREATE TABLE IF NOT EXISTS players (
      telegram_id   TEXT PRIMARY KEY,
      telegram_name TEXT,
      wallet_address TEXT UNIQUE,
      registered_at INTEGER DEFAULT (unixepoch()),
      notify_fill   INTEGER DEFAULT 1,
      notify_reveal INTEGER DEFAULT 1,
      notify_result INTEGER DEFAULT 1
    );

    CREATE TABLE IF NOT EXISTS games (
      game_id       INTEGER PRIMARY KEY,
      mode_id       INTEGER NOT NULL,
      status        TEXT NOT NULL DEFAULT 'open',
      player_count  INTEGER NOT NULL DEFAULT 0,
      max_players   INTEGER NOT NULL,
      entry_fee_pls TEXT NOT NULL,
      reveal_deadline INTEGER,
```

```
    winner_address  TEXT,
    runner_address  TEXT,
    cashx_bought    TEXT,
    cashx_burned    TEXT,
    created_at    INTEGER DEFAULT (unixepoch()),
    updated_at    INTEGER DEFAULT (unixepoch())
  );

  CREATE TABLE IF NOT EXISTS game_players (
    game_id       INTEGER NOT NULL,
    wallet_address TEXT NOT NULL,
    slot_index    INTEGER NOT NULL,
    has_revealed  INTEGER DEFAULT 0,
    joined_at     INTEGER DEFAULT (unixepoch()),
    PRIMARY KEY (game_id, wallet_address),
    FOREIGN KEY (game_id) REFERENCES games(game_id)
  );

  CREATE TABLE IF NOT EXISTS secrets (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    telegram_id   TEXT NOT NULL,
    game_id       INTEGER NOT NULL,
    secret_enc    TEXT NOT NULL,
    salt_enc      TEXT NOT NULL,
    commitment    TEXT NOT NULL,
    saved_at      INTEGER DEFAULT (unixepoch()),
    UNIQUE (telegram_id, game_id),
    FOREIGN KEY (game_id) REFERENCES games(game_id)
  );

  CREATE TABLE IF NOT EXISTS notifications (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    telegram_id   TEXT NOT NULL,
    game_id       INTEGER,
    type          TEXT NOT NULL,
    sent_at       INTEGER DEFAULT (unixepoch())
  );

CREATE TABLE IF NOT EXISTS referrals (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  referrer_id   TEXT NOT NULL,
  referee_id    TEXT NOT NULL UNIQUE,
  bonus_paid    INTEGER DEFAULT 0,
  bonus_amount  TEXT DEFAULT '0',
  created_at    INTEGER DEFAULT (unixepoch())
);
```

```
CREATE TABLE IF NOT EXISTS referral_earnings (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  referrer_id   TEXT NOT NULL,
  referee_id    TEXT NOT NULL,
  deposit_amount TEXT NOT NULL,
  bonus_amount   TEXT NOT NULL,
  tx_note        TEXT,
  created_at     INTEGER DEFAULT (unixepoch())
);

CREATE INDEX IF NOT EXISTS idx_referrals_referrer ON referrals(referrer_id);
CREATE INDEX IF NOT EXISTS idx_referral_earnings_referrer ON referral_earnings(

CREATE TABLE IF NOT EXISTS meta (
  key        TEXT PRIMARY KEY,
  value      TEXT NOT NULL,
  updated_at INTEGER DEFAULT (unixepoch())
);

CREATE TABLE IF NOT EXISTS accounts (
  telegram_id     TEXT PRIMARY KEY,
  cashx_balance   TEXT NOT NULL DEFAULT '0',
  pls_balance     TEXT NOT NULL DEFAULT '0',
  total_deposited TEXT NOT NULL DEFAULT '0',
  total_withdrawn TEXT NOT NULL DEFAULT '0',
  total_wagered   TEXT NOT NULL DEFAULT '0',
  total_won       TEXT NOT NULL DEFAULT '0',
  created_at      INTEGER DEFAULT (unixepoch()),
  updated_at      INTEGER DEFAULT (unixepoch())
);

CREATE TABLE IF NOT EXISTS account_deposit_addresses (
  telegram_id     TEXT PRIMARY KEY,
  deposit_address TEXT NOT NULL UNIQUE,
  created_at      INTEGER DEFAULT (unixepoch())
);

CREATE TABLE IF NOT EXISTS processed_deposits (
  tx_hash    TEXT PRIMARY KEY,
  created_at INTEGER DEFAULT (unixepoch())
);

CREATE INDEX IF NOT EXISTS idx_acct_addr ON account_deposit_addresses(deposit_a

CREATE TABLE IF NOT EXISTS account_transactions (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  telegram_id   TEXT NOT NULL,
```

```
  type          TEXT NOT NULL,
  amount        TEXT NOT NULL,
  token         TEXT NOT NULL DEFAULT 'cashx',
  game          TEXT,
  game_id       INTEGER,
  tx_hash       TEXT,
  note          TEXT,
  created_at    INTEGER DEFAULT (unixepoch())
);

CREATE TABLE IF NOT EXISTS withdrawal_requests (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  telegram_id   TEXT NOT NULL,
  amount        TEXT NOT NULL,
  to_address    TEXT NOT NULL,
  status        TEXT NOT NULL DEFAULT 'pending',
  tx_hash       TEXT,
  created_at    INTEGER DEFAULT (unixepoch()),
  updated_at    INTEGER DEFAULT (unixepoch())
);

CREATE INDEX IF NOT EXISTS idx_acct_tx_telegram ON account_transactions(telegra
CREATE INDEX IF NOT EXISTS idx_withdrawal_telegram ON withdrawal_requests(teleg
CREATE INDEX IF NOT EXISTS idx_withdrawal_status ON withdrawal_requests(status)

  CREATE TABLE IF NOT EXISTS deposit_sessions (
    deposit_address TEXT PRIMARY KEY,
    telegram_id     TEXT NOT NULL,
    game_id         INTEGER NOT NULL,
    mode_id         INTEGER NOT NULL,
    entry_fee_wei   TEXT NOT NULL,
    source_address  TEXT,
    status          TEXT NOT NULL DEFAULT 'pending',
    commit_tx       TEXT,
    reveal_tx       TEXT,
    created_at      INTEGER DEFAULT (unixepoch()),
    updated_at      INTEGER DEFAULT (unixepoch())
  );

  CREATE INDEX IF NOT EXISTS idx_game_players_wallet  ON game_players(wallet_ad
  CREATE INDEX IF NOT EXISTS idx_secrets_telegram     ON secrets(telegram_id);
  CREATE INDEX IF NOT EXISTS idx_notifs_telegram      ON notifications(telegram
  CREATE INDEX IF NOT EXISTS idx_deposit_telegram     ON deposit_sessions(teleg
  CREATE INDEX IF NOT EXISTS idx_deposit_game         ON deposit_sessions(game_
  CREATE INDEX IF NOT EXISTS idx_deposit_status       ON deposit_sessions(statu
`);
```

```
  return Promise.resolve();
}

// ── Players ──────────────────────────────────────────────

function upsertPlayer({ telegramId, telegramName, walletAddress }) {
  return db.prepare(`
    INSERT INTO players (telegram_id, telegram_name, wallet_address)
    VALUES (?, ?, ?)
    ON CONFLICT(telegram_id) DO UPDATE SET
      telegram_name   = excluded.telegram_name,
      wallet_address  = COALESCE(excluded.wallet_address, wallet_address)
  `).run(telegramId, telegramName, walletAddress || null);
}

function getPlayerByWallet(walletAddress) {
  return db.prepare(
    'SELECT * FROM players WHERE LOWER(wallet_address) = LOWER(?)'
  ).get(walletAddress);
}

function getPlayerByTelegram(telegramId) {
  return db.prepare('SELECT * FROM players WHERE telegram_id = ?').get(telegramId
}

function setWallet(telegramId, walletAddress) {
  return db.prepare(
    'UPDATE players SET wallet_address = ? WHERE telegram_id = ?'
  ).run(walletAddress, telegramId);
}

function updateNotifyPrefs(telegramId, { fill, reveal, result }) {
  return db.prepare(`
    UPDATE players SET
      notify_fill   = COALESCE(?, notify_fill),
      notify_reveal = COALESCE(?, notify_reveal),
      notify_result = COALESCE(?, notify_result)
    WHERE telegram_id = ?
  `).run(
    fill   !== undefined ? (fill   ? 1 : 0) : null,
    reveal !== undefined ? (reveal ? 1 : 0) : null,
    result !== undefined ? (result ? 1 : 0) : null,
    telegramId
  );
}

// ── Games ────────────────────────────────────────────────
```

```
function upsertGame({ gameId, modeId, status, playerCount, maxPlayers, entryFeePl
  return db.prepare(`
    INSERT INTO games (game_id, mode_id, status, player_count, max_players, entry
    VALUES (?, ?, ?, ?, ?, ?)
    ON CONFLICT(game_id) DO UPDATE SET
      status        = excluded.status,
      player_count  = excluded.player_count,
      updated_at    = unixepoch()
  `).run(gameId, modeId, status, playerCount, maxPlayers, entryFeePls);
}

function updateGameStatus(gameId, status, extra = {}) {
  const { revealDeadline, winnerAddress, runnerAddress, cashxBought, cashxBurned
  return db.prepare(`
    UPDATE games SET
      status           = ?,
      reveal_deadline  = COALESCE(?, reveal_deadline),
      winner_address   = COALESCE(?, winner_address),
      runner_address   = COALESCE(?, runner_address),
      cashx_bought     = COALESCE(?, cashx_bought),
      cashx_burned     = COALESCE(?, cashx_burned),
      updated_at       = unixepoch()
    WHERE game_id = ?
  `).run(
    status,
    revealDeadline || null,
    winnerAddress  || null,
    runnerAddress  || null,
    cashxBought    || null,
    cashxBurned    || null,
    gameId
  );
}

function getGame(gameId) {
  return db.prepare('SELECT * FROM games WHERE game_id = ?').get(gameId);
}

function getOpenGames() {
  return db.prepare("SELECT * FROM games WHERE status IN ('open','filled') ORDER
}

// ── Game players ──────────────────────────────────────

function addGamePlayer(gameId, walletAddress, slotIndex) {
  return db.prepare(`
```

```
    INSERT OR IGNORE INTO game_players (game_id, wallet_address, slot_index)
    VALUES (?, ?, ?)
  `).run(gameId, walletAddress.toLowerCase(), slotIndex);
}

function markRevealed(gameId, walletAddress) {
  return db.prepare(`
    UPDATE game_players SET has_revealed = 1
    WHERE game_id = ? AND LOWER(wallet_address) = LOWER(?)
  `).run(gameId, walletAddress);
}

function getGamePlayers(gameId) {
  return db.prepare('SELECT * FROM game_players WHERE game_id = ? ORDER BY slot_i
}

// ─── Secrets (server-side encrypted storage) ──────────

/**
 * Encrypt a secret/salt before storing.
 * Uses AES-256-GCM with a per-record IV.
 * SECRET_ENCRYPTION_KEY must be a 32-byte hex string in .env
 */
function encrypt(plaintext) {
  if (!SECRET_KEY) throw new Error('SECRET_ENCRYPTION_KEY not set in .env');
  const key = Buffer.from(SECRET_KEY, 'hex');
  const iv  = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const enc  = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]);
  const tag  = cipher.getAuthTag();
  return iv.toString('hex') + ':' + tag.toString('hex') + ':' + enc.toString('hex
}

function decrypt(ciphertext) {
  if (!SECRET_KEY) throw new Error('SECRET_ENCRYPTION_KEY not set in .env');
  const key = Buffer.from(SECRET_KEY, 'hex');
  const [ivHex, tagHex, encHex] = ciphertext.split(':');
  const iv  = Buffer.from(ivHex,  'hex');
  const tag = Buffer.from(tagHex, 'hex');
  const enc = Buffer.from(encHex, 'hex');
  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  return decipher.update(enc) + decipher.final('utf8');
}

function saveSecret({ telegramId, gameId, secret, salt, commitment }) {
  return db.prepare(`
```

```
    INSERT INTO secrets (telegram_id, game_id, secret_enc, salt_enc, commitment)
    VALUES (?, ?, ?, ?, ?)
    ON CONFLICT(telegram_id, game_id) DO UPDATE SET
      secret_enc  = excluded.secret_enc,
      salt_enc    = excluded.salt_enc,
      commitment  = excluded.commitment
  `).run(telegramId, gameId, encrypt(secret), encrypt(salt), commitment);
}

function getSecret(telegramId, gameId) {
  const row = db.prepare(
    'SELECT * FROM secrets WHERE telegram_id = ? AND game_id = ?'
  ).get(telegramId, gameId);
  if (!row) return null;
  return {
    ...row,
    secret: decrypt(row.secret_enc),
    salt:   decrypt(row.salt_enc),
  };
}




// ── Referral system ──────────────────────────────────

const REFERRAL_BONUS_PCT = 5; // 5% of first deposit

function registerReferral(referrerId, refereeId) {
  // Ignore self-referral
  if (referrerId === refereeId) return null;
  // Only register once per referee
  return db.prepare(`
    INSERT OR IGNORE INTO referrals (referrer_id, referee_id) VALUES (?, ?)
  `).run(referrerId, refereeId);
}

function getReferrer(refereeId) {
  const row = db.prepare('SELECT referrer_id FROM referrals WHERE referee_id = ?'
  return row ? row.referrer_id : null;
}

function payReferralBonus(refereeId, depositAmountWei) {
  const referrerId = getReferrer(refereeId);
  if (!referrerId) return null;

  // Check if already paid (first deposit only)
```

```
  const existing = db.prepare(
    'SELECT bonus_paid FROM referrals WHERE referee_id = ?'
  ).get(refereeId);
  if (!existing || existing.bonus_paid) return null;

  const deposit  = BigInt(depositAmountWei);
  const bonus    = (deposit * BigInt(REFERRAL_BONUS_PCT)) / 100n;
  if (bonus === 0n) return null;

  // Credit referrer
  db.prepare(`UPDATE accounts SET cashx_balance = CAST(CAST(cashx_balance AS INTE
    .run(bonus.toString(), referrerId);

  // Mark bonus paid
  db.prepare(`UPDATE referrals SET bonus_paid = 1, bonus_amount = ? WHERE referee
    .run(bonus.toString(), refereeId);

  // Log earning
  db.prepare(`INSERT INTO referral_earnings (referrer_id, referee_id, deposit_amo
    .run(referrerId, refereeId, depositAmountWei.toString(), bonus.toString(), `5

  // Log transaction for referrer
  db.prepare(`INSERT INTO account_transactions (telegram_id, type, amount, token,
    .run(referrerId, 'credit', bonus.toString(), 'cashx', `Referral bonus from us

  return { referrerId, bonus };
}

function getReferralStats(telegramId) {
  const count = db.prepare('SELECT COUNT(*) as c FROM referrals WHERE referrer_id
  const earned = db.prepare('SELECT COALESCE(SUM(CAST(bonus_amount AS INTEGER)),0
  const { ethers } = require('ethers');
  return {
    referralCount: count ? count.c : 0,
    totalEarned:   ethers.formatEther(BigInt(earned ? earned.total : 0)),
  };
}

// ─── Meta key/value store ──────────────────────────

function getMetaValue(key) {
  const row = db.prepare('SELECT value FROM meta WHERE key = ?').get(key);
  return row ? row.value : null;
}

function setMetaValue(key, value) {
  return db.prepare(`
```

```
    INSERT INTO meta (key, value) VALUES (?, ?)
    ON CONFLICT(key) DO UPDATE SET value = excluded.value, updated_at = unixepoch
  `).run(key, value);
}

// ─── Account / Balance ──────────────────────────────────

function getOrCreateAccount(telegramId) {
  db.prepare(`
    INSERT OR IGNORE INTO accounts (telegram_id) VALUES (?)
  `).run(telegramId);
  return db.prepare('SELECT * FROM accounts WHERE telegram_id = ?').get(telegramI
}

function getBalance(telegramId) {
  const acc = getOrCreateAccount(telegramId);
  return {
    cashx: BigInt(acc.cashx_balance || '0'),
    pls:   BigInt(acc.pls_balance   || '0'),
  };
}

function creditBalance(telegramId, amountWei, token = 'cashx', { game, gameId, tx
  const acc = getOrCreateAccount(telegramId);
  const field = token === 'cashx' ? 'cashx_balance' : 'pls_balance';
  const current = BigInt(acc[field] || '0');
  const newBal   = current + BigInt(amountWei);

  db.prepare(`UPDATE accounts SET ${field} = ?, total_deposited = ?, updated_at =
    .run(newBal.toString(), (BigInt(acc.total_deposited||'0') + BigInt(amountWei)

  db.prepare(`INSERT INTO account_transactions (telegram_id, type, amount, token,
    VALUES (?, 'credit', ?, ?, ?, ?, ?, ?)`)
    .run(telegramId, amountWei.toString(), token, game||null, gameId||null, txHas

  return newBal;
}

function debitBalance(telegramId, amountWei, token = 'cashx', { game, gameId, not
  const acc = getOrCreateAccount(telegramId);
  const field = token === 'cashx' ? 'cashx_balance' : 'pls_balance';
  const current = BigInt(acc[field] || '0');
  const amount  = BigInt(amountWei);

  if (current < amount) throw new Error('Insufficient balance');

  const newBal = current - amount;
```

```
  db.prepare(`UPDATE accounts SET ${field} = ?, total_wagered = ?, updated_at = u
    .run(newBal.toString(), (BigInt(acc.total_wagered||'0') + amount).toString(),

  db.prepare(`INSERT INTO account_transactions (telegram_id, type, amount, token,
    VALUES (?, 'debit', ?, ?, ?, ?, ?)`)
    .run(telegramId, amount.toString(), token, game||null, gameId||null, note||nu

  return newBal;
}

function hasSufficientBalance(telegramId, amountWei, token = 'cashx') {
  const bal = getBalance(telegramId);
  const field = token === 'cashx' ? 'cashx' : 'pls';
  return bal[field] >= BigInt(amountWei);
}

function getTransactionHistory(telegramId, limit = 20) {
  return db.prepare(
    'SELECT * FROM account_transactions WHERE telegram_id = ? ORDER BY created_at
  ).all(telegramId, limit);
}

function createWithdrawalRequest(telegramId, amountWei, toAddress) {
  // Debit balance first
  debitBalance(telegramId, amountWei, 'cashx', { note: `Withdrawal to ${toAddress

  return db.prepare(`
    INSERT INTO withdrawal_requests (telegram_id, amount, to_address)
    VALUES (?, ?, ?)
  `).run(telegramId, amountWei.toString(), toAddress);
}

function updateWithdrawalStatus(id, status, txHash = null) {
  return db.prepare(`
    UPDATE withdrawal_requests SET status = ?, tx_hash = ?, updated_at = unixepoc
    WHERE id = ?
  `).run(status, txHash, id);
}

function getPendingWithdrawals() {
  return db.prepare(
    "SELECT * FROM withdrawal_requests WHERE status = 'pending' ORDER BY created_
  ).all();
}


// ── Deposit sessions ──────────────────────────────
```

```
function upsertDepositSession({ telegramId, gameId, modeId, depositAddress, entry
  return db.prepare(`
    INSERT INTO deposit_sessions (deposit_address, telegram_id, game_id, mode_id,
    VALUES (?, ?, ?, ?, ?, ?)
    ON CONFLICT(deposit_address) DO UPDATE SET
      status     = excluded.status,
      updated_at = unixepoch()
  `).run(depositAddress, telegramId, gameId, modeId, entryFeeWei, status);
}

function updateDepositStatus(depositAddress, status, txHash = null) {
  return db.prepare(`
    UPDATE deposit_sessions SET
      status     = ?,
      commit_tx  = CASE WHEN ? IS NOT NULL AND status = 'pending' THEN ? ELSE com
      reveal_tx  = CASE WHEN ? IS NOT NULL AND status = 'committed' THEN ? ELSE r
      updated_at = unixepoch()
    WHERE deposit_address = ?
  `).run(status, txHash, txHash, txHash, txHash, depositAddress);
}

function updateDepositSourceAddress(depositAddress, sourceAddress) {
  return db.prepare(
    'UPDATE deposit_sessions SET source_address = ?, updated_at = unixepoch() WHE
  ).run(sourceAddress, depositAddress);
}

function getDepositSession(depositAddress) {
  return db.prepare('SELECT * FROM deposit_sessions WHERE deposit_address = ?').g
}

function getDepositSessionsByGame(gameId) {
  return db.prepare('SELECT * FROM deposit_sessions WHERE game_id = ?').all(gameI
}

function getDepositSessionsByTelegram(telegramId) {
  return db.prepare(
    'SELECT * FROM deposit_sessions WHERE telegram_id = ? ORDER BY created_at DES
  ).all(telegramId);
}

function getPendingDepositSessions() {
  return db.prepare(
    "SELECT * FROM deposit_sessions WHERE status IN ('pending','committed')"
  ).all();
}
```

```
// ─── Notifications log ──────────────────────────────────

function logNotification(telegramId, gameId, type) {
  return db.prepare(
    'INSERT INTO notifications (telegram_id, game_id, type) VALUES (?, ?, ?)'
  ).run(telegramId, gameId, type);
}

function wasNotified(telegramId, gameId, type) {
  const row = db.prepare(
    'SELECT 1 FROM notifications WHERE telegram_id = ? AND game_id = ? AND type =
  ).get(telegramId, gameId, type);
  return !!row;
}


// ─── Account addresses ──────────────────────────────────

function getAccountDepositAddress(telegramId) {
  const row = db.prepare('SELECT deposit_address FROM account_deposit_addresses W
  return row ? row.deposit_address : null;
}

function setAccountDepositAddress(telegramId, address) {
  return db.prepare(`
    INSERT OR IGNORE INTO account_deposit_addresses (telegram_id, deposit_address
    VALUES (?, ?)
  `).run(telegramId, address.toLowerCase());
}

function getAllAccountAddresses() {
  return db.prepare('SELECT telegram_id, deposit_address FROM account_deposit_add
}

function isDepositProcessed(txHash) {
  return !!db.prepare('SELECT 1 FROM processed_deposits WHERE tx_hash = ?').get(t
}

function markDepositProcessed(txHash) {
  return db.prepare('INSERT OR IGNORE INTO processed_deposits (tx_hash) VALUES (?
}

// ─── Exports ────────────────────────────────────────────

module.exports = {
  init,
```

```
  upsertPlayer,
  getPlayerByWallet,
  getPlayerByTelegram,
  setWallet,
  updateNotifyPrefs,
  upsertGame,
  updateGameStatus,
  getGame,
  getOpenGames,
  addGamePlayer,
  markRevealed,
  getGamePlayers,
  saveSecret,
  getSecret,
  registerReferral,
  getReferrer,
  payReferralBonus,
  getReferralStats,
  getMetaValue,
  setMetaValue,
  getOrCreateAccount,
  getAccountDepositAddress,
  setAccountDepositAddress,
  getAllAccountAddresses,
  isDepositProcessed,
  markDepositProcessed,
  getBalance,
  creditBalance,
  debitBalance,
  hasSufficientBalance,
  getTransactionHistory,
  createWithdrawalRequest,
  updateWithdrawalStatus,
  getPendingWithdrawals,
  upsertDepositSession,
  updateDepositStatus,
  updateDepositSourceAddress,
  getDepositSession,
  getDepositSessionsByGame,
  getDepositSessionsByTelegram,
  getPendingDepositSessions,
  logNotification,
  wasNotified,
};
```